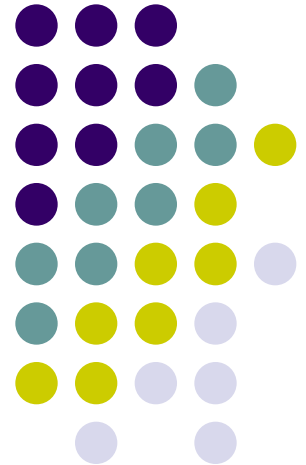


php



Dr:Wafaa Alnbhany

Lecture 7



PHP \$GLOBALS

\$GLOBALS is a PHP super global variable which is used to access global variables from anywhere in the PHP script (also from within functions or methods).

PHP stores all global variables in an array called \$GLOBALS[*index*]. The *index* holds the name of the variable.

The example below shows how to use the super global variable \$GLOBALS:

Example

```
<?php
$x = 75;
$y = 25;

function addition() {
    $GLOBALS['z'] = $GLOBALS['x'] + $GLOBALS['y'];
}

addition();
echo $z;
?>
```

Definition and Usage

The `static` keyword is used to declare properties and methods of a class as static. Static properties and methods can be used without creating an instance of the class.

The `static` keyword is also used to declare variables in a function which keep their value after the function has ended.

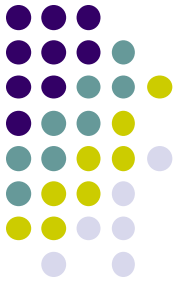
Example

Use a static variable in a function:

```
<?php
function add1() {
    static $number = 0;
    $number++;
    return $number;
}

echo add1();
echo "<br>";
echo add1();
echo "<br>";
echo add1();
?>
```

The output



1

2

3

Example

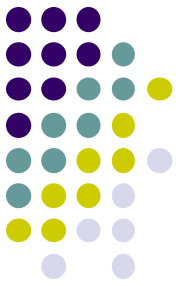
Create and use static properties and methods:

```
<?php
class MyClass {
    public static $str = "Hello World!";

    public static function hello() {
        echo MyClass::$str;
    }
}

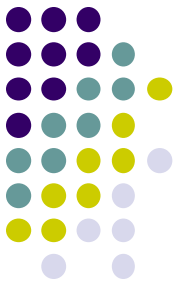
echo MyClass::$str;
echo "<br>";
echo MyClass::hello();
?>
```

The output



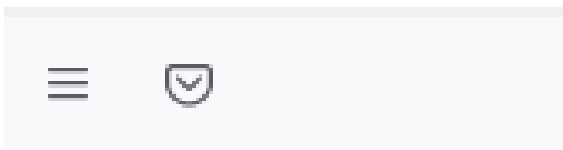
Hello World!

Hello World!



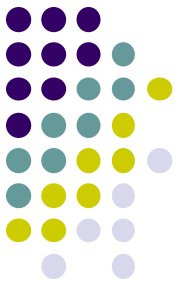
```
<?php
    $x=5;
?>
<?php
    echo $x;
?>
```

The output



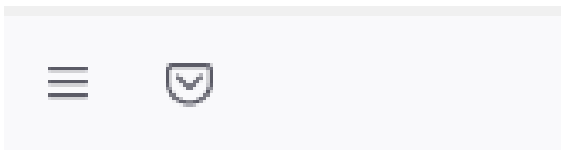
5

php variable



```
<?php
$x=5;
?>
<?php
echo $x;
?>
```

The output



5

PHP \$GLOBALS

\$GLOBALS is a PHP super global variable which is used to access global variables from anywhere in the PHP script (also from within functions or methods).

PHP stores all global variables in an array called \$GLOBALS[*index*]. The *index* holds the name of the variable.

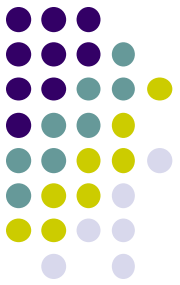
The example below shows how to use the super global variable \$GLOBALS:

Example

```
<?php
$x = 75;
$y = 25;

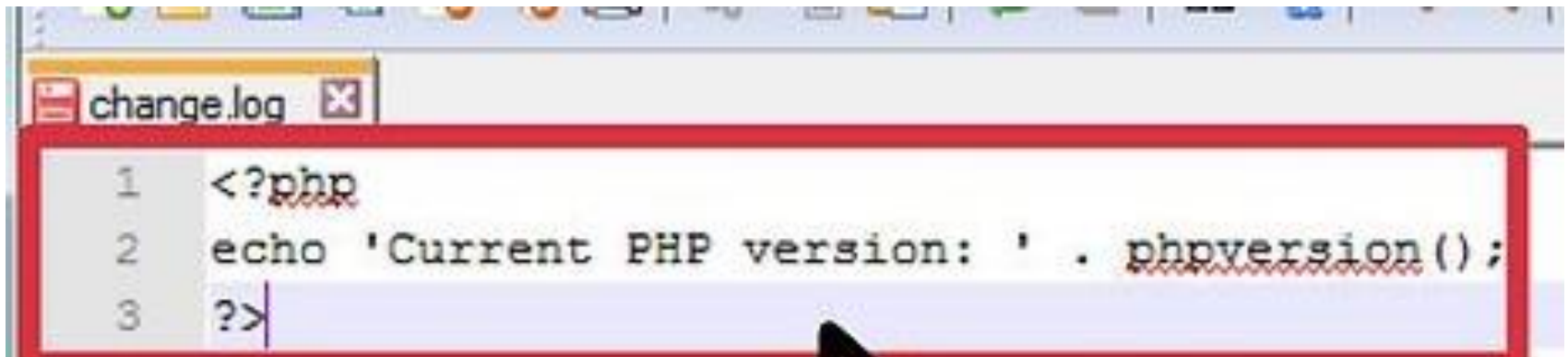
function addition() {
    $GLOBALS['z'] = $GLOBALS['x'] + $GLOBALS['y'];
}

addition();
echo $z;
?>
```



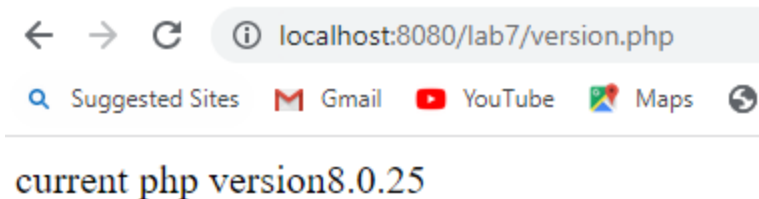
PHP version

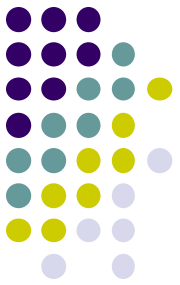
- `Phpversion()` :- function to know PHP version



```
1 <?php
2 echo 'Current PHP version: ' . phpversion();
3 ?>
```

The output

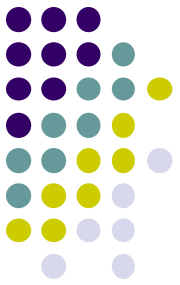




PHP

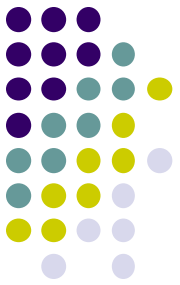
OOP - Classes and Objects

- A class is a template for objects, and an object is an instance of class.



OOP Case

- Let's assume we have a class named Fruit. A Fruit can have properties like name, color, weight, etc. We can define variables like \$name, \$color, and \$weight to hold the values of these properties.
- When the individual objects (apple, banana, etc.) are created, they inherit all the properties and behaviors from the class, but each object will have different values for the properties.



Define a Class

A class is defined by using the **class**

keyword, followed by the name of the class and a pair of curly braces (`{}`). All its properties and methods go inside the braces:

Note:

In a class, variables are called properties and functions are called methods!

Below we declare a class named `Fruit` consisting of two properties (`$name` and `$color`) and two methods `set_name()` and `get_name()` for setting and getting the `$name` property:

```
<?php
```

```
class Fruit {
```

```
    // Properties
```

```
    public $name;
```

```
    public $color;
```

```
    // Methods
```

```
    function set_name($name) {
```

```
        $this->name = $name;
```

```
    }
```

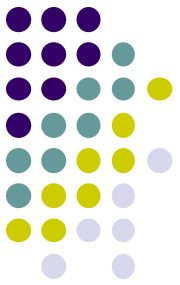
```
    function get_name() {
```

```
        return $this->name;
```

```
    }
```

```
}
```

```
?>
```





Define Objects

Classes are nothing without objects! We can create multiple objects from a class. Each object has all the properties and methods defined in the class, but they will have different property values.

Objects of a class are created using the

new

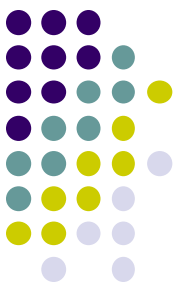
keyword.

```
<?php
class Fruit {
    // Properties
    public $name;
    public $color;

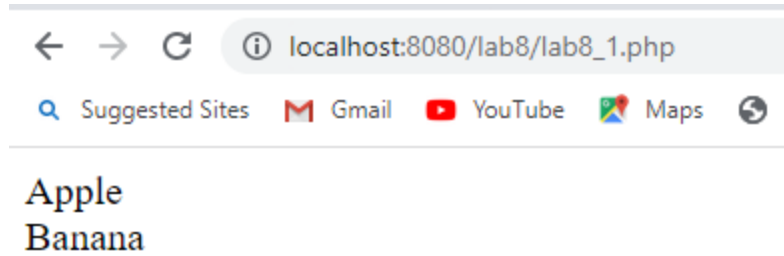
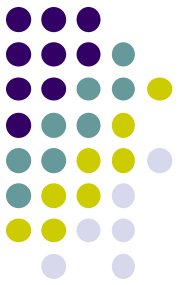
    // Methods
    function set_name($name) {
        $this->name = $name;
    }
    function get_name() {
        return $this->name;
    }
}

$apple = new Fruit();
$banana = new Fruit();
$apple->set_name('Apple');
$banana->set_name('Banana');

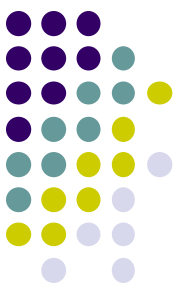
echo $apple->get_name();
echo "<br>";
echo $banana->get_name();
?>
```



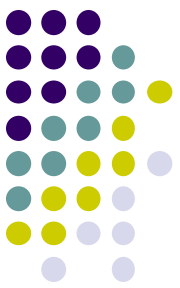
The output



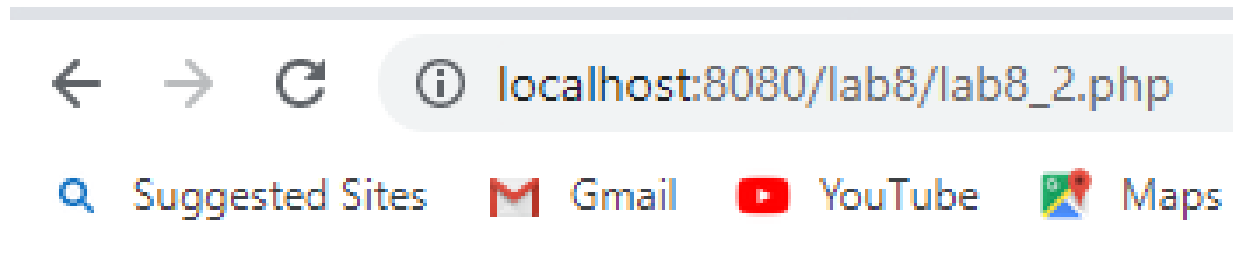
In the example below, we add two more methods to class Fruit, for setting and getting the \$color property:



```
<?php
class
Fruit {
    // Properties
    public $name;
    public $color;
    // Methods
    function set_name($name) {
        $this->name = $name;
    }
    function get_name() {
        return $this->name;
    }
    function set_color($color) {
        $this->color = $color;
    }
    function get_color() {
        return $this->color;
    } }
$apple =new Fruit();
$apple->set_name( 'Apple');
$apple->set_color('Red');
echo"Name: ". $apple->get_name();
echo"<br>";
echo"Color: ". $apple->get_color();
?>
```

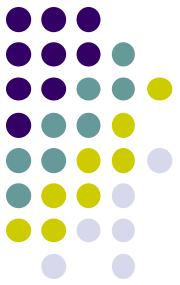


The output



Name: Apple
Color: Red

example

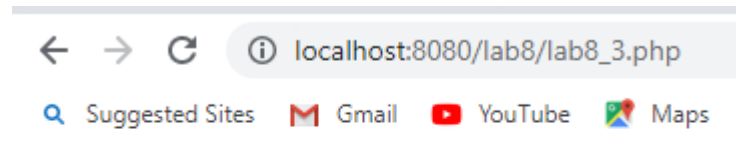
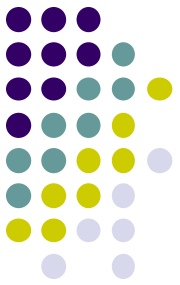


```
<?php
Class A
{
    public $myname;    //this is a member variable of this class

    function callme() {
        $myname = 'function variable';
        $this->myname = 'Member variable';
        echo $myname;
        echo "<br>";
        echo $this->myname;
    }
}

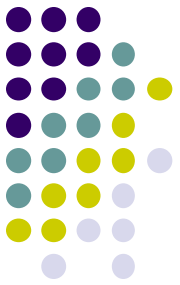
$a =new A();
$a->callme();
?>
```

The output



function variable
Member variable

Using this



\$this is mainly used to refer properties of a class.

Example:

Class A

```
{  
    public $myname;    //this is a member variable of this class
```

```
function callme() {  
    $myname = 'function variable';  
    $this->myname = 'Member variable';  
    echo $myname;  
    echo $this->myname;  
}  
}
```

output:

function variable

member variable

Definition and Usage

The `static` keyword is used to declare properties and methods of a class as static. Static properties and methods can be used without creating an instance of the class.

The `static` keyword is also used to declare variables in a function which keep their value after the function has ended.

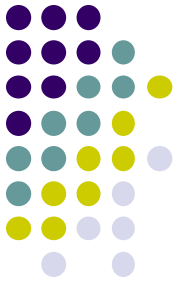
Example

Use a static variable in a function:

```
<?php
function add1() {
    static $number = 0;
    $number++;
    return $number;
}

echo add1();
echo "<br>";
echo add1();
echo "<br>";
echo add1();
?>
```

The output

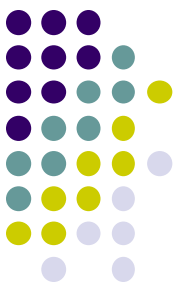


1

2

3

OOP - Static property



:: that allows access to static, constant, and overridden properties or methods of a class.

Example

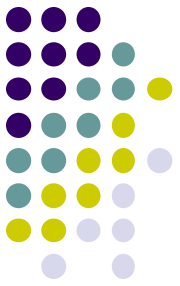
Create and use static properties and methods:

```
<?php
class MyClass {
    public static $str = "Hello World!";

    public static function hello() {
        echo MyClass::$str;
    }
}

echo MyClass::$str;
echo "<br>";
echo MyClass::hello();
?>
```

The output



Hello World!

Hello World!

Static property Example

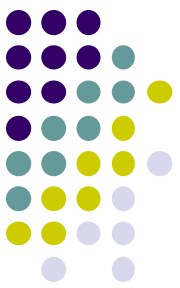


```
<?php
Class B
{
    public static $myname= " is a static variable";// public static variable

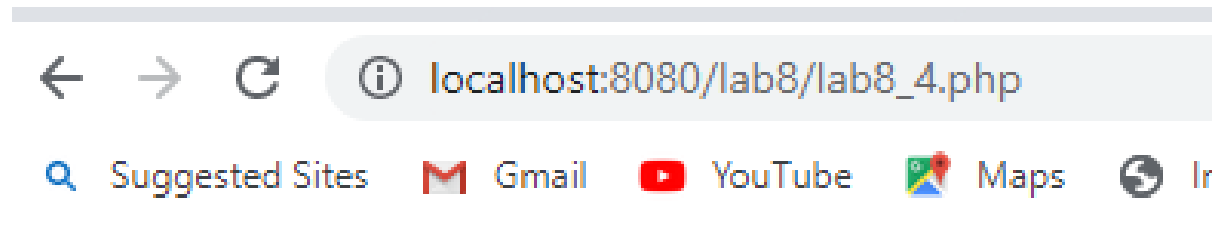
    public function callme() { // public function
        $m = 'function variable';
        $this->m = 'Member variable';
        echo $m;
        echo "<br>";
        echo $this->m;
        echo "<br>";
    }

    public static function s(){ // public static function
        echo B::$myname;
    }
}

$a =new B;
$a->callme();
echo B::$myname;
echo "<br>";
echo B::s();
?>
```

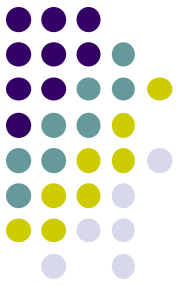


The Output



function variable
Member variable
is a static variable
is a static variable

H.W



- 1. What is the difference between a **public** and a **static** function?
- 2. Write a simple PHP class that contains a **static function** which displays the following string:
“**The first class**”
- 3. Write a simple PHP class that contains a **function** which displays the following string:
“**The second class**”